



Конспект по теме «Репликация и масштабирование. Часть 1»

Содержание:

1. Репликация
2. Репликация master-slave
3. Репликация master-master
4. Настройка и тестирование master-slave репликации (MySQL)
5. Настройка и тестирование master-master репликации (MySQL)

1. Репликация

Репликация — это процесс копирования данных с одного источника на другой, обеспечивающий доступность и согласованность данных между несколькими базами данных. Репликация используется для улучшения производительности, отказоустойчивости и масштабируемости.



Важно отметить, что **репликация не является резервным копированием**. Случайное удаление данных на основном сервере приведёт к их удалению на всех репликах. Резервные копии необходимо создавать отдельно.

Виды репликации:

- **Синхронная:** основной сервер подтверждает операцию записи только после того, как все реплики подтвердят получение данных. Этот метод гарантирует согласованность данных, но замедляет работу, так как требует ожидания ответа от всех реплик.
- **Асинхронная:** основной сервер подтверждает запись сразу, а данные на реплики копируются в фоновом режиме. Это обеспечивает высокую скорость, но допускает риск временной рассинхронизации данных.

2. Репликация master-slave

Это наиболее распространённая схема, в которой есть один главный сервер и один или несколько подчинённых.

В этой конфигурации:

- **Master** (ведущий) — основной сервер. Все операции записи (INSERT, UPDATE, DELETE) происходят только на нём.
- **Slave** (ведомый) — копия мастера. Он получает все изменения от мастера и обслуживает запросы на чтение (SELECT).

Принцип работы:

1. Master записывает все изменения в специальный журнал — binary log.
2. Slave копирует этот журнал к себе в relay log.
3. Slave применяет изменения из своего журнала, обновляя собственные данные.

Эта схема эффективна для систем, где количество операций чтения значительно превышает количество операций записи.

Типы репликации:

- **покомандная** — репликация SQL-запросов
- **построчная** — репликация изменённых данных (строк)

3. Репликация master-master

В этой конфигурации все серверы равноправны — каждый является одновременно и мастером, и слейвом для другого. Запросы на запись или чтение можно отправлять на любой из серверов, и изменения будут распространены на все остальные.

Такая схема повышает отказоустойчивость: если один из мастеров выйдет из строя, система продолжит работу, используя оставшиеся серверы.

4. Настройка и тестирование master-slave репликации (MySQL)

1. Создание двух экземпляров MySQL

Для начала вам нужно создать два экземпляра MySQL (или PostgreSQL), которые будут выполнять роль мастера и слейва.

Пример настройки в Docker:

Для настройки с использованием Docker, вам нужно создать два контейнера, один для мастера и один для слейва.

Пример конфигурации файлов:

- **Dockerfile для мастера (MySQL)**

```
FROM mysql:8.0
COPY ./master.cnf /etc/mysql/conf.d/my.cnf
COPY ./master.sql /docker-entrypoint-initdb.d/start.sql
ENV MYSQL_ROOT_PASSWORD=rootpassword
CMD ["mysqld"]
```

master.sql — скрипт для создания пользователя для репликации:

```
CREATE USER 'repl'@'%' IDENTIFIED BY 'slavepass';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';
FLUSH PRIVILEGES;
```

- **Dockerfile для слейва (MySQL):**

```
FROM mysql:8.0
COPY ./slave.cnf /etc/mysql/conf.d/my.cnf
COPY ./slave.sql /docker-entrypoint-initdb.d/start.sql
ENV MYSQL_ROOT_PASSWORD=rootpassword
CMD ["mysqld"]
```

slave.sql — скрипт для подключения слейва к мастеру:

```
CHANGE REPLICATION SOURCE TO  
SOURCE_HOST='mysql_master', -- Имя хоста мастера  
SOURCE_USER='repl', -- Имя пользователя  
SOURCE_PASSWORD='slavepass', -- Пароль этого пользователя  
SOURCE_SSL=1;  
START REPLICA;
```

2. Создание конфигурации для репликации

- Для мастера в файле конфигурации нужно указать уникальный **server-id** и включить бинарный лог **log-bin**.

master.cnf — конфигурационный файл для мастера:

```
[mysqld]  
server-id = 1  
log-bin = mysql-bin  
binlog-format = ROW
```

- Для слейва нужно указать его уникальный **server-id**, включить режим «только для чтения» с помощью **read-only**, чтобы предотвратить записи на слейве.

slave.cnf — конфигурационный файл для слейва:

```
[mysqld]  
server-id = 2  
read-only = 1
```

3. Настройка сети Docker

3.1. Собираем образы

Сначала из двух Docker-файлов делаем готовые образы для слейва и мастера:

```
docker build -t mysql_slave -f ./Dockerfile_slave .  
docker build -t mysql_master -f ./Dockerfile_master .
```

3.2. Создаём сеть

Дальше выделяем собственную сеть:

```
docker network create replication .
```

3.3. Запускаем контейнеры

Мастер и слейв подключаем к одной сети и сразу пробрасываем разные хост-порты:

```
docker run --name mysql_master --net replication -p 3306:3306 mysql_master  
docker run --name mysql_slave --net replication -p 3307:3306 mysql_slave
```

4. Тестирование

После настройки репликации протестируйте работу:

- Создайте базу данных на мастере:

```
CREATE DATABASE test_db;  
  
USE test_db;  
  
CREATE TABLE test_table (id INT PRIMARY KEY, name VARCHAR(50));  
  
INSERT INTO test_table VALUES (1, 'Master Record');
```

- Проверьте на слейве, что данные появились:

```
SHOW DATABASES;  
  
USE test_db;  
  
SHOW TABLES;  
  
SELECT * FROM test_table;
```

Если все настроено правильно, данные, созданные на мастере, должны отобразиться на слейве.

4. Настройка и тестирование master-master репликации (MySQL)

1. Создание двух экземпляров MySQL

Для настройки репликации master-master создаются два экземпляра MySQL, оба из которых работают как мастера и слейвы одновременно.

Пример Dockerfile для master-master (MySQL):

- **Dockerfile для первого мастера (Master-1):**

```
FROM mysql:8.0  
  
COPY ./master-1.cnf /etc/mysql/conf.d/my.cnf  
  
COPY ./master-1.sql /docker-entrypoint-initdb.d/start.sql  
  
ENV MYSQL_ROOT_PASSWORD=rootpassword  
  
CMD ["mysqld"]
```

master-1.sql — создание пользователя для репликации:

```
CREATE USER 'repl_1'@'%' IDENTIFIED BY 'pass';  
  
GRANT REPLICATION SLAVE ON *.* TO 'repl_1'@'%';  
  
CHANGE REPLICATION SOURCE TO  
  
    SOURCE_HOST='master-2',  
  
    SOURCE_USER='repl_2',  
  
    SOURCE_PASSWORD='pass',  
  
    SOURCE_SSL=1;  
  
START REPLICA;
```

- **Dockerfile для второго мастера (Master-2):**

```
FROM mysql:8.0  
  
COPY ./master-2.cnf /etc/mysql/conf.d/my.cnf  
  
COPY ./master-2.sql /docker-entrypoint-initdb.d/start.sql  
  
ENV MYSQL_ROOT_PASSWORD=rootpassword  
  
CMD ["mysqld"]
```

master-2.sql — создание пользователя для репликации:

```
CREATE USER 'repl_1'@'%' IDENTIFIED BY 'pass';  
  
GRANT REPLICATION SLAVE ON *.* TO 'repl_1'@'%';  
  
CHANGE REPLICATION SOURCE TO  
  
    SOURCE_HOST='master-2',  
  
    SOURCE_USER='repl_2',  
  
    SOURCE_PASSWORD='pass',  
  
    SOURCE_SSL=1;  
  
START REPLICA;
```

2. Создание конфигурации для репликации

Конфигурационные файлы **master-1.cnf** и **master-2.cnf** различаются только значением `server_id`.

master-1.cnf:

```
[mysqld]
server-id = 1
log-bin = mysql-bin
binlog-format = ROW
```

master-2.cnf:

```
[mysqld]
server-id = 2
log-bin = mysql-bin
binlog-format = ROW
```

3. Запуск контейнеров

Как и в случае с master-slave, создайте сеть Docker и запустите два контейнера:

```
docker network create replication_mm
docker run --name master-1 --net replication_mm -p 3306:3306 -d master-1
docker run --name master-2 --net replication_mm -p 3307:3306 -d master-2
```


4. Тестирование:

После настройки протестируйте работу:

- Создайте запись на первом мастере:

```
CREATE DATABASE test_db;  
  
USE test_db;  
  
CREATE TABLE test_table (id INT PRIMARY KEY, name VARCHAR(50));  
  
INSERT INTO test_table VALUES (1, 'Master-1 Record');
```

- Проверьте на втором мастере, что запись появилась:

```
SHOW DATABASES;  
  
USE test_db;  
  
SHOW TABLES;  
  
SELECT * FROM test_table;
```

- Создайте запись на втором мастере:

```
INSERT INTO test_table VALUES (2, 'Master-2 Record');
```

Проверьте на первом мастере, что запись появилась:

```
SELECT * FROM test_table;
```

Если репликация настроена правильно, вы должны увидеть данные, записанные на одном сервере и на другом.